

NNGraph DSL Compiler

A Domain-Specific Language for Describing Neural Network Architectures
and Automatic PyTorch Code Generation

Project Technical Report

June 30, 2026

Contents

Abstract	3
1 Introduction	4
1.1 Motivation	4
1.2 Goal	5
1.3 Scope	5
2 Related Work	6
2.1 The NADER Framework	6
2.2 ANTLR4	6
2.2.1 Listener Pattern vs. Visitor Pattern	6
2.3 AST Utilities and Visualizations	7
2.3.1 Why AST Is Not Used Here	7
3 Language Design	8
3.1 Program Structure	8
3.2 Grammar	9
3.3 Type System	10
3.4 Layer Types	10
3.4.1 Parameterized Layers	10
3.4.2 Activations	11
3.4.3 Graph Operations (No Module)	11
3.4.4 Universal Parameters	12
3.5 Edge Syntax	12
3.6 Complete Example: MLP	12
4 Implementation	15
4.1 Compiler Architecture	15
4.1.1 File Responsibilities	16
4.2 Semantic Analyzer	16
4.2.1 Value Propagation in the Tree	16

4.2.2	Validation Rules	17
4.2.3	Error Message Format	17
4.2.4	_FakeCtx for Post-Walk Errors	18
4.2.5	Listener Execution Flow	18
4.3	Code Generator	23
4.4	Variable Assignment Algorithm	23
4.4.1	MLP Trace (Linear Chain)	24
4.4.2	ResBlock Trace (Fan-Out Then Merge)	24
4.4.3	InceptionBlock Trace (Two Parallel Branches)	25
4.4.4	TransformerEncoder Trace (Two Implicit Residuals)	25
4.4.5	Special Code Generation Cases	26
4.4.6	Output Code Structure	27
4.5	Shape Inference	27
4.6	Graphviz Visualization	28
4.7	Training Scaffold	28
4.8	Quantization Annotations	28
4.9	ONNX Export	28
5	Evaluation	30
5.1	Test Suite	30
5.2	Forward Pass Results	31
5.3	Variable Correctness Proof	31
5.4	Codebase Statistics	32
6	Design Decisions	33
6.1	Listener vs. Visitor	33
6.2	load_from_listener() vs. generate(traversal)	33
6.3	Variable Naming with node_id	33
7	Conclusion and Future Work	34
7.1	Conclusion	34
7.2	Implemented Features	34
7.3	Future Work	34
	References	35

Abstract

NNGraph DSL is a domain-specific language designed to describe neural network architectures as directed graphs. It accepts `.nng` files as input and produces executable PyTorch code consisting of a complete `nn.Module` subclass with `__init__` and `forward` methods.

The compiler is built with ANTLR4 version 4.13.2 and uses the Listener pattern for semantic analysis. Kahn's topological sort algorithm is employed both in the semantic analyzer (cycle detection) and in the code generator (execution ordering). Tensor shape inference, Graphviz graph visualization, training scaffolding, quantization annotations, and ONNX export are also supported.

The test suite comprises 55 tests across four files. Four example architectures—MLP, ResBlock, InceptionBlock, and TransformerEncoder—compile successfully and produce correct output tensor shapes in their forward pass.

Introduction

1.1 Motivation

Writing `nn.Module` subclasses in PyTorch by hand for complex architectures is repetitive and error-prone. For a network with ten layers, the programmer must:

- Define each layer separately with correct parameters in `__init__`
- Manually manage the execution order in `forward`
- Name intermediate variables in branching topologies
- Ensure no shared tensor is overwritten before it is consumed

Consider the following PyTorch code that must be written manually:

Listing 1.1.1 ▶ Manual boilerplate for a ResBlock

```
1 class ResBlock(nn.Module):
2     def __init__(self):
3         super(ResBlock, self).__init__()
4         self.conv1 = nn.Conv2d(
5             in_channels=64, out_channels=64,
6             kernel_size=3, stride=1, padding=1)
7         self.bn1 = nn.BatchNorm2d(num_features=64)
8         self.relu1 = nn.ReLU()
9         # ... and so on for each layer
10
11     def forward(self, x):
12         identity = x
13         x = self.conv1(x)
14         x = self.bn1(x)
15         x = self.relu1(x)
16         # ... manage residual connection manually
```

```
17         x = x + identity
18     return x
```

With NNGraph DSL, the architecture is described once as a graph and the compiler generates this code automatically.

1.2 Goal

The goal of NNGraph DSL is to make the `.nng` file the single source of truth for the architecture. The programmer describes the graph; the compiler generates all boilerplate.

1.3 Scope

This tool is suited for describing static architectures—networks whose computational graph is known at model definition time. Support for dynamic control flow within **forward** is planned for future versions.

Related Work

2.1 The NADER Framework

Definition 2.1.1 ▶ NADER

NADER (Neural Architecture Design via Representation) is a framework that models neural network architectures as directed computational graphs (DAGs). Each node represents a layer operation and each edge represents tensor flow.

NNGraph DSL adopts this graph model as the foundation of its language design.

2.2 ANTLR4

Definition 2.2.1 ▶ ANTLR4

ANTLR4 (ANother Tool for Language Recognition) is a tool that automatically generates a lexer, parser, and listener/visitor from a `.g4` grammar file. Version used: ANTLR 4.13.2.

2.2.1 Listener Pattern vs. Visitor Pattern

ANTLR4 provides two patterns for traversing the parse tree:

- **Listener:** `ParseTreeWalker` automatically traverses the tree and calls `enter*/exit*` methods. No return values are needed.
- **Visitor:** The programmer controls traversal and each method returns a value.

This compiler uses the Listener pattern. The `exit*` methods in `custom_listener.py` accumulate state into `self.nodes` and `self.edges`—this approach is more natural for building a graph model from a parse tree.

2.3 AST Utilities and Visualizations

This compiler incorporates abstract syntax tree (AST) utilities to support optional structure inspections and visualizations. Three helper files are used:

- `ast.py` — AST and `TreeNode` classes
- `make_ast_subtree.py` — parse tree node composition
- `ast_to_networkx_graph.py` — optional graph visualization

2.3.1 Why AST Is Not Used Here

Most traditional compilers represent programs as abstract syntax trees (ASTs). Expression-based languages—where operators have precedence and scopes nest hierarchically—naturally map to tree structures.

NNGraph DSL, however, describes neural network architectures as *directed acyclic graphs* (DAGs). Nodes represent layer operations; edges represent tensor flow. The graph structure is the domain model, not the parse tree.

The compiler performs multiple graph-based operations that require adjacency lists and explicit edge relationships:

- **Topological sort** (Kahn’s algorithm) for execution ordering in `forward()`
- **Cycle detection** via topological sort (DAG validation)
- **Reachability analysis** via BFS from input to output
- **Residual connection validation** (checking in-degree of merge nodes)
- **Shape inference** by propagating tensor shapes along edges
- **Graphviz DOT export** for architecture visualization
- **ONNX export** (ONNX uses a graph format)

An AST would encode the parse tree hierarchy (`model_block`, `graph_block`, `node_decl`), but it would obscure the computational graph that the language is designed to express. For this reason, the semantic analyzer stores nodes and edges directly in dictionaries and lists—the natural representation for graph algorithms.

The AST construction code (`exitEveryRule`, `exitStart`) remains in `custom_listener.py` for optional debugging visualization via `--show-ast`. It is not used during compilation.

Language Design

3.1 Program Structure

Every `.nng` file consists of three blocks. The `config` block is optional:

Listing 3.1.1 ▶ General structure of a `.nng` file

```

1  model <Name> {
2      input  <id> : tensor(<shape>)
3      output <id>
4  }
5
6  graph {
7      node <id> : <LayerType>(<params>)
8      ...
9      edge <src> -> <dst>
10     edge <src> -> <dst> [label="<string>"]
11     ...
12 }
13
14 config {
15     batch_size = <int>
16     device      = "cpu" | "cuda"
17     loss        = "<LossFn>"           // enables training
18     ⇨ scaffold
19     optimizer   = "<Optimizer>"       // Adam | SGD | AdamW |
20     ⇨ RMSprop
21     lr          = <float>              // learning rate
22     epochs      = <int>                // training epochs
23 }
```

Definition 3.1.2 ▶ Model block

The **model** block specifies the generated class name, the input node identifier and tensor shape, and the output node identifier.

Definition 3.1.3 ▶ Graph block

The **graph** block defines all layer nodes and directed edges. Edge declaration order does not matter; the compiler computes execution order from the graph structure.

Definition 3.1.4 ▶ Config block

The **config** block sets values for the `__main__` block of the generated code. **batch_size** defaults to 1 and **device** defaults to 'cpu'. If the **loss** key is set, the code generator produces a complete training loop with optimizer and loss function.

3.2 Grammar

Definition 3.2.1 ▶ NNGraph.g4 statistics

- Grammar file: **NNGraph.g4** (56 lines)
- Parser rules: 14 rules (snake_case)
- Lexer tokens: 20 tokens (ALL_CAPS)

Complete grammar rules:

Listing 3.2.2 ▶ NNGraph.g4 grammar

```

1  grammar NNGraph;
2  start      : model_block graph_block config_block? EOF;
3  model_block : MODEL ID '{' input_decl output_decl '}';
4  input_decl  : INPUT ID ':' TENSOR shape_expr;
5  output_decl : OUTPUT ID;
6  graph_block : GRAPH '{' (node_decl | edge_decl)* '}';
7  node_decl   : NODE ID ':' layer_expr;
8  layer_expr  : ID '(' param_list? ')';
9  param_list  : param (',' param)*;
10 param       : ID '=' value;

```

```

11 edge_decl    : EDGE ID ARROW ID ('[' LABEL '=' STRING ']')?;
12 config_block: CONFIG '{' config_entry* '}';
13 config_entry: ID '=' value;
14 value        : FLOAT_LITERAL | INT_LITERAL | BOOL_LITERAL
15              | STRING | NONE | shape_expr;
16 shape_expr   : '(' INT_LITERAL (',' INT_LITERAL)* ')';

```

Important design note: `input_decl` uses `TENSOR shape_expr` rather than `TENSOR '(' shape_expr ')'` because `shape_expr` itself includes the parentheses. This common mistake in early grammar versions was corrected.

3.3 Type System

DSL Type	Python Type	Example
<code>int</code>	<code>int</code>	<code>128</code>
<code>float</code>	<code>float</code>	<code>0.1</code>
<code>bool</code>	<code>bool</code>	<code>true / false</code>
<code>string</code>	<code>str</code>	<code>"relu"</code>
<code>shape</code>	<code>tuple[int]</code>	<code>(64, 32, 32)</code>
<code>None</code>	<code>NoneType</code>	<code>None</code>

The distinction between `int` and `float` is enforced at compile time. For example, `Dropout(p=1)` produces an error because `p` expects a `float`, not an `int`.

3.4 Layer Types

3.4.1 Parameterized Layers

These layers generate a `self.<id> = nn.X(...)` line in `__init__`.

DSL Keyword	PyTorch Class	Key Parameters
Linear	<code>nn.Linear</code>	<code>in_features</code> , <code>out_features</code> , <code>bias</code>
Conv2d	<code>nn.Conv2d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code> , <code>padding</code>
Conv1d	<code>nn.Conv1d</code>	<code>in_ch</code> , <code>out_ch</code> , <code>kernel</code> , <code>stride</code>
BatchNorm2d	<code>nn.BatchNorm2d</code>	<code>num_features</code>
LayerNorm	<code>nn.LayerNorm</code>	<code>normalized_shape</code>
MaxPool2d	<code>nn.MaxPool2d</code>	<code>kernel</code> , <code>stride</code>
AvgPool2d	<code>nn.AvgPool2d</code>	<code>kernel</code> , <code>stride</code>
Dropout	<code>nn.Dropout</code>	<code>p</code>
Flatten	<code>nn.Flatten</code>	<code>start_dim</code> , <code>end_dim</code>
Embedding	<code>nn.Embedding</code>	<code>num_embeddings</code> , <code>embedding_dim</code>
MultiHeadAttn	<code>nn.MultiheadAttention</code>	<code>embed_dim</code> , <code>num_heads</code>
LSTM	<code>nn.LSTM</code>	<code>input_size</code> , <code>hidden_size</code> , <code>num_layers</code>
GRU	<code>nn.GRU</code>	<code>input_size</code> , <code>hidden_size</code>

3.4.2 Activations

`ReLU`, `Sigmoid`, `Tanh`, `GELU`, `Softmax(dim)`, `LeakyReLU(negative_slope)`, `ELU(alpha)`.

3.4.3 Graph Operations (No Module)

These types do not generate a line in `__init__`:

Keyword	Parameters	Generated Code
Add	—	<code>out = a + b</code>
Concat	<code>dim: int</code>	<code>out = torch.cat([a,b], dim=d)</code>
Residual	—	<code>out = main + shortcut</code>
Split	<code>chunks: int</code> , <code>dim: int</code>	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>

3.4.4 Universal Parameters

The **quant** parameter can be applied to any node regardless of layer type. Valid values: "dynamic", "static", "qat".

Listing 3.4.1 ▶ Quantization annotation example

```
1 node fc1 : Linear(in_features=784, out_features=128,  
  ↪ quant="dynamic")
```

3.5 Edge Syntax

Definition 3.5.1 ▶ Simple edge

```
edge src -> dst
```

Defines a directed edge from node **src** to node **dst**.

Definition 3.5.2 ▶ Labeled edge

```
edge src -> dst [label="name"]
```

Defines an edge with a string label. Labels serve two roles:

- "shortcut": marks the skip connection edge into a **Residual()** node.
- "residual_*": marks a skip connection into a node with two inputs (Transformer pattern).

3.6 Complete Example: MLP

The .nng file for a three-layer network:

Listing 3.6.1 ▶ mlp.nng

```
1 model MLP {  
2     input x : tensor(784)  
3     output out  
4 }  
5  
6 graph {
```

```

7     node fc1    : Linear(in_features=784, out_features=256)
8     node relu1  : ReLU()
9     node fc2    : Linear(in_features=256, out_features=128)
10    node relu2  : ReLU()
11    node drop   : Dropout(p=0.3)
12    node fc3    : Linear(in_features=128, out_features=10)
13    node out    : Softmax(dim=1)
14
15    edge x      -> fc1
16    edge fc1    -> relu1
17    edge relu1  -> fc2
18    edge fc2    -> relu2
19    edge relu2  -> drop
20    edge drop   -> fc3
21    edge fc3    -> out
22  }
23
24  config {
25      batch_size = 64
26      device = "cuda"
27  }

```

Generated PyTorch code:

Listing 3.6.2 ▶ Generated PyTorch code for MLP

```

1  import torch
2  import torch.nn as nn
3
4
5  class MLP(nn.Module):
6      def __init__(self):
7          super(MLP, self).__init__()
8          self.fc1 = nn.Linear(in_features=784,
9                               ↪ out_features=256)
9          self.relu1 = nn.ReLU()

```

```
10         self.fc2 = nn.Linear(in_features=256,  
11                               ↪ out_features=128)  
11         self.relu2 = nn.ReLU()  
12         self.drop = nn.Dropout(p=0.3)  
13         self.fc3 = nn.Linear(in_features=128,  
14                               ↪ out_features=10)  
14         self.out = nn.Softmax(dim=1)  
15  
16     def forward(self, x):  
17         x = self.fc1(x)  
18         x = self.relu1(x)  
19         x = self.fc2(x)  
20         x = self.relu2(x)  
21         x = self.drop(x)  
22         x = self.fc3(x)  
23         x = self.out(x)  
24         return x  
25  
26  
27 if __name__ == '__main__':  
28     device = torch.device('cuda')  
29     model = MLP().to(device)  
30     x = torch.randn(64, 784).to(device)  
31     print(model(x).shape)
```

Implementation

4.1 Compiler Architecture

Definition 4.1.1 ▶ Compilation pipeline

The compiler has six stages:

1. **Parsing:** `FileStream` $\rightarrow$ `NNGraphLexer` $\rightarrow$ `NNGraphParser.program()`
 $\rightarrow$ parse tree
2. **Semantic analysis:** `ParseTreeWalker` + `NNGraphCustomListener` $\rightarrow$
graph model
3. **Topological sort:** compute execution order using Kahn's algorithm
4. **Shape inference:** propagate tensor shapes through the graph and detect dimension mismatches (optional: `--show-shapes`)
5. **Code generation:** `CodeGenerator.generate()` $\rightarrow$ Python string
6. **Output:** write to `.py` file, Graphviz DOT output, or ONNX export

4.1.1 File Responsibilities

File	Role
NNGraph.g4	Grammar source of truth (56 lines)
gen/NNGraphLexer.py	Generated by ANTLR4
gen/NNGraphParser.py	Generated by ANTLR4
gen/NNGraphListener.py	Base listener — should not be edited
custom_listener.py	Semantic analysis
code_generator.py	PyTorch code generation
shape_inference.py	Compile-time tensor shape inference
graph_viz.py	Graphviz DOT visualization
onnx_export.py	ONNX model export
main.py	CLI interface and <code>compile_nng()</code>

4.2 Semantic Analyzer

Definition 4.2.1 ▶ NNGraphCustomListener

The class `NNGraphCustomListener(NNGraphListener)` in `custom_listener.py` implements all semantic analysis logic. Internal state:

- `self.nodes`: `dict[id, {type, params, line}]`
- `self.edges`: `list[{src, dst, label, line}]`
- `self.config`: `dict[key, value]`

4.2.1 Value Propagation in the Tree

The `exitShape_expr` and `exitValue` methods store values on the context object itself:

Listing 4.2.2 ▶ Value propagation via `ctx.pvalue` in `exitShape_expr`

```

1 def exitShape_expr(self, ctx):
2     ints = [int(ctx.INT_LITERAL(i).getText())
3             for i in range(ctx.INT_LITERAL().__len__())]
```

```

4     ctx.pvalue = tuple(ints)
5     ctx.ptype  = tuple

```

This approach ensures that each context, after exiting, has its children's values available.

4.2.2 Validation Rules

Eight validation rules are executed in two phases:

Phase one (during tree walk):

1. Duplicate node ID: in `exitNode_decl`
2. Invalid edge reference: in `exitEdge_decl`
3. Unknown layer type: in `exitNode_decl`
4. Parameter type mismatch: in `exitNode_decl`

Phase two (in `exitGraph_block`, after graph completion):

5. Undeclared output node: in `_validate_output_declared`
6. Orphan node: in `_validate_orphans`
7. Residual arity $\neq$ 2: in `_validate_residual_arity`
8. Cycle detection via Kahn's algorithm: in `_validate_no_cycles`
9. BFS reachability: in `_validate_reachability`

4.2.3 Error Message Format

Listing 4.2.3 ▶ Error format

```

1  [line L:C] SemanticError: <message>
2  [line L] ShapeError at '<node>': <message>

```

L is the line number (1-based) and **C** is the token column (0-based). All error conditions halt compilation and exit with code 1.

4.2.4 `_FakeCtx` for Post-Walk Errors

Errors raised in `exitGraph_block` do not have a live context. An internal class `_FakeCtx` carries the line number from the node's declaration record:

Listing 4.2.4 ▶ `_FakeCtx` class for preserving line numbers

```
1 class _FakeCtx:
2     class start:
3         line    = info["line"]
4         column  = 0
```

4.2.5 Listener Execution Flow

The `ParseTreeWalker.walk(listener, parse_tree)` method performs a depth-first traversal of the parse tree. For each node, it calls `enterX(ctx)` before visiting children and `exitX(ctx)` after all children have been visited.

Definition 4.2.5 ▶ Walker invocation in `main.py`

```
1 listener = NNGraphCustomListener()
2 listener.rule_names = parser.ruleNames
3 walker = ParseTreeWalker()
4 walker.walk(listener, parse_tree)
```

The custom listener overrides only `exit*` methods. All `enter*` methods remain as no-ops from the base `NNGraphListener` class.

Attribute Propagation (Bottom-Up)

Values flow upward through the tree via context attributes:

Listing 4.2.6 ▶ `exitValue` stores parsed value in `ctx.pvalue`

```
1 def exitValue(self, ctx):
2     if ctx.FLOAT_LITERAL():
3         ctx.pvalue = float(ctx.FLOAT_LITERAL().getText())
4         ctx.ptype  = float
5     elif ctx.INT_LITERAL():
```

```

6         ctx.pvalue = int(ctx.INT_LITERAL().getText())
7         ctx.ptype  = int
8     elif ctx.BOOL_LITERAL():
9         ctx.pvalue = ctx.BOOL_LITERAL().getText() == 'true'
10        ctx.ptype  = bool
11    # ... other cases

```

Parent rules read child attributes:

Listing 4.2.7 ▶ `exitParam` reads `ctx.value().pvalue` from child

```

1  def exitParam(self, ctx):
2      ctx.param_name = ctx.ID().getText()
3      ctx.param_value = ctx.value().pvalue
4      ctx.param_type  = ctx.value().ptype

```

Semantic Extraction and Detailed Method Behavior

The custom listener relies on a set of `exit*` methods to process the parse tree bottom-up. Below is the detailed explanation of how all listener functions operate:

- **`exitShape_expr(ctx)`**: Parses list of dimensions from `INT_LITERAL` tokens. It builds a Python `tuple` and binds it to `ctx.pvalue`, setting `ctx.ptype` to `tuple`.
- **`exitValue(ctx)`**: Evaluates literal values (`FLOAT_LITERAL`, `INT_LITERAL`, `BOOL_LITERAL`, `STRING`, `NONE`, or nested `shape_expr`). It converts the matched text into its corresponding Python representation, attaching the result to `ctx.pvalue` and the type to `ctx.ptype`.
- **`exitModel_block(ctx)`**: Extracts the neural network model identifier via `ctx.ID().getText()` and stores it in `self.model_name`.
- **`exitInput_decl(ctx)`**: Stores the input variable identifier in `self.input_id` and propagates the parsed shape to `self.input_shape`. Additionally, it populates `self.nodes` with a pseudo-node of type `"__input__"` at the current line number, enabling downstream edge verification to validate connections from the input.
- **`exitOutput_decl(ctx)`**: Saves the output variable identifier in `self.output_id` to track the network's terminal node.

- **exitParam(ctx)**: Processes key-value parameter configurations. It maps the parameter identifier to `ctx.param_name`, and associates the evaluated values and types to `ctx.param_value` and `ctx.param_type` respectively.
- **exitLayer_expr(ctx)**: Processes layer constructor configurations. It identifies the layer class name from the ID and constructs a dictionary mapping parameter names to their parsed metadata. The dictionary is attached to `ctx.params`.
- **exitNode_decl(ctx)**: Registers nodes and validates structural constraints. It ensures that the node identifier has not been previously declared. It matches the layer type against `LAYER_SCHEMA`, verifies that every declared parameter matches the schema specifications (types and formats), validates that optional quantization annotations (`quant`) use a valid mode, and registers the validated node into `self.nodes`.
- **exitEdge_decl(ctx)**: Processes connections between nodes. It extracts the source and destination identifiers, validates that both endpoints exist in `self.nodes`, checks for optional label annotations (e.g., "shortcut" or "residual_*"), and registers the edge in `self.edges`.
- **exitConfig_entry(ctx)**: Extracts compiler configuration overrides (such as target device or batch_size) and populates the `self.config` dictionary.
- **exitGraph_block(ctx)**: Triggers after the graph block has been fully walked. It runs five sequential validation procedures: ensuring the output node is declared, verifying that no nodes are orphaned, checking residual input arity, verifying the absence of directed cycles, and validating path reachability from input to output.
- **exitEveryRule(ctx)**: Intercepts all rule exits. If rule names are loaded, it invokes `make_ast_subtree` to build and merge abstract syntax tree nodes bottom-up.
- **exitStart(ctx)**: Finalizes AST construction, binding the complete AST hierarchy under the root rule to the listener's AST collector.

Graph Validation Algorithms

`exitGraph_block` runs five validation passes after the graph is fully constructed:

1. Orphan Detection (`_validate_orphans`):

Listing 4.2.8 ▶ Orphan detection algorithm

```

1 referenced = set()
2 for e in self.edges:
3     referenced.add(e["src"])
4     referenced.add(e["dst"])
5 for node_id, info in self.nodes.items():
6     if info["type"] == "__input__":
7         continue
8     if node_id not in referenced:
9         error(f"Node '{node_id}' is declared but never
           ↪ referenced.")

```

Collects all node IDs appearing in edges. Any declared node (except input) not in this set is orphaned—declared but never connected.

2. Cycle Detection (`_validate_no_cycles`):

Uses Kahn's topological sort algorithm. A DAG can be fully sorted; if any nodes remain with `in_degree > 0` after processing, a cycle exists.

Listing 4.2.9 ▶ Kahn's algorithm for cycle detection

```

1 in_degree = dict.fromkeys(self.nodes, 0)
2 adj       = {n: [] for n in self.nodes}
3 for e in self.edges:
4     adj[e["src"]].append(e["dst"])
5     in_degree[e["dst"]] += 1
6
7 queue     = deque(n for n, d in in_degree.items() if d == 0)
8 visited   = 0
9 while queue:
10    node = queue.popleft()
11    visited += 1
12    for nxt in adj[node]:
13        in_degree[nxt] -= 1
14        if in_degree[nxt] == 0:
15            queue.append(nxt)

```

```

16
17 if visited != len(self.nodes):
18     remaining = [n for n, d in in_degree.items() if d > 0]
19     error(f"Cycle detected involving nodes: {remaining}.")

```

3. Reachability Analysis (`_validate_reachability`):

Performs BFS from the input node. Every declared node and the output node must be reachable.

Listing 4.2.10 ▸ BFS reachability check

```

1 adj = {n: [] for n in self.nodes}
2 for e in self.edges:
3     adj[e["src"]].append(e["dst"])
4
5 visited = set()
6 queue = deque([self.input_id])
7 while queue:
8     node = queue.popleft()
9     if node in visited:
10        continue
11    visited.add(node)
12    for nxt in adj[node]:
13        queue.append(nxt)
14
15 for node_id, info in self.nodes.items():
16     if node_id not in visited and info["type"] !=
17         ↪ "__input__":
18         error(f"Node '{node_id}' is not reachable from
19             ↪ input.")
20
21 if self.output_id not in visited:
22     error(f"Output node '{self.output_id}' is not
23         ↪ reachable.")

```

Catches disconnected subgraphs and broken input-to-output paths.

Cycle detection and reachability are independent checks. A graph can be acyclic yet fail reachability (disconnected components), or be connected yet contain cycles. Both properties are required for a valid neural network DAG.

4.3 Code Generator

Definition 4.3.1 ▶ CodeGenerator

The `CodeGenerator` class in `code_generator.py` has four internal steps:

1. `_compute_graph()`: build adjacency lists and topological sort
2. `_assign_vars()`: assign a Python variable name to each node
3. `_emit_init()` + `_emit_forward()`: generate code lines
4. `_assemble()`: combine into final Python source

4.4 Variable Assignment Algorithm

Definition 4.4.1 ▶ Core variable assignment invariant

`var_at[n] = 'x'` only when `out_degree[predecessor] == 1`.

When a node fans out to multiple destinations (`out_degree > 1`), its tensor is still needed by other branches. Overwriting `x` at this point would corrupt it. Instead, the downstream node receives its own identifier as a variable name.

The four complete rules in `_assign_vars`:

Listing 4.4.2 ▶ Variable assignment rules in `_assign_vars`

```

1 Rule 1 - Input node:
2     var_at[input_id] = 'x'
3
4 Rule 2 - Merge node (in_degree > 1):
5     var_at[n] = 'x'           if out_degree[n] <= 1
6     var_at[n] = node_id      if out_degree[n] > 1
7
8 Rule 3 - Branch start (predecessor.out_degree > 1):
9     var_at[n] = node_id

```



```

10
11 Rule 4 - Linear continuation (predecessor.out_degree == 1):
12     var_at[n] = var_at[predecessor]

```

4.4.1 MLP Trace (Linear Chain)

Example 4.4.3 ▶ Variable trace for MLP

Topology: `x -> fc1 -> relu1 -> fc2 -> relu2 -> drop -> fc3 -> out`

Every predecessor has `out_degree == 1` $\Rightarrow$ Rule 4 always applies $\Rightarrow$ all nodes have `var_at = 'x'`. The `forward` method uses `x` everywhere.

4.4.2 ResBlock Trace (Fan-Out Then Merge)

Example 4.4.4 ▶ Variable trace for ResBlock

`x` fans out to `conv1` and `skip` (shortcut). Therefore `out_degree[x] = 2`.

- `conv1`: Rule 3 $\Rightarrow$ `var_at = 'conv1'`
- `bn1`, `relu1`, `conv2`, `bn2`: Rule 4 $\Rightarrow$ all `'conv1'`
- `skip` (Residual, `in_degree=2`): Rule 2, `out_degree=1` $\Rightarrow$ `var_at = 'x'`

Generated code:

Listing 4.4.5 ▶ Generated forward pass for ResBlock

```

1 conv1 = self.conv1(x)
2 conv1 = self.bn1(conv1)
3 conv1 = self.relu1(conv1)
4 conv1 = self.conv2(conv1)
5 conv1 = self.bn2(conv1)
6 x     = conv1 + x      # Residual node
7 x     = self.flatten(x)

```

4.4.3 InceptionBlock Trace (Two Parallel Branches)

Example 4.4.6 ▶ Variable trace for InceptionBlock

out_degree[x] = 2 (to convA1 and convB1).

- convA1: Rule 3 $\Rightarrow$ 'convA1'
- convB1: Rule 3 $\Rightarrow$ 'convB1'
- convA2: Rule 4 $\Rightarrow$ inherits 'convA1'
- convB2: Rule 4 $\Rightarrow$ inherits 'convB1'
- cat1 (Concat, in_degree=2): Rule 2 $\Rightarrow$ 'x'

Generated code:

Listing 4.4.7 ▶ Generated forward pass for InceptionBlock

```

1 convA1 = self.convA1(x)
2 convA1 = self.convA2(convA1)
3 convB1 = self.convB1(x)
4 convB1 = self.convB2(convB1)
5 x      = torch.cat([convA1, convB1], dim=1)  # Concat
        ↪ node
6 x      = self.bn(x)
7 x      = self.out(x)

```

4.4.4 TransformerEncoder Trace (Two Implicit Residuals)

Example 4.4.8 ▶ Variable trace for TransformerEncoder

out_degree[x] = 2 and out_degree[norm1] = 2.

- attn: Rule 3 $\Rightarrow$ 'attn'
- drop1: Rule 4 $\Rightarrow$ 'attn'
- norm1 (in_deg=2, out_deg=2): Rule 2 $\Rightarrow$ 'norm1'
- ff1: Rule 3 $\Rightarrow$ 'ff1'
- gelu1, drop2, ff2: Rule 4 $\Rightarrow$ 'ff1'
- norm2 (in_deg=2, out_deg=1): Rule 2 $\Rightarrow$ 'x'

Generated code:

Listing 4.4.9 ▶ Generated forward pass for TransformerEncoder

```

1  attn, _ = self.attn(x, x, x)    # MultiHeadAttn
   ↪ triple-pass
2  attn     = self.drop1(attn)
3  norm1    = self.norm1(x + attn) # implicit residual_1
4  ff1      = self.ff1(norm1)
5  ff1      = self.gelu1(ff1)
6  ff1      = self.drop2(ff1)
7  ff1      = self.ff2(ff1)
8  x        = self.norm2(norm1 + ff1) # implicit
   ↪ residual_2
9  x        = self.out(x)
10 x        = self.flatten(x)
11 return x

```

4.4.5 Special Code Generation Cases

Pattern	Condition	Generated Code
MultiHeadAttn	Layer type	<code>out, _ = self.attn(x, x, x)</code>
LSTM / GRU	Layer type	<code>out, _ = self.lstm(x)</code>
Residual	No module	<code>out = main + shortcut</code>
Add	No module	<code>out = a + b</code>
Concat	No module	<code>out = torch.cat([a,b], dim=d)</code>
Split	No module	<code>split_0, split_1, ... = torch.chunk(x,n,dim=d)</code>
Implicit residual	<code>residual_*</code> label	<code>out = self.norm(skip + main)</code>

Parameter name mapping: The DSL uses shorter names. The code generator maps them to PyTorch names: `in_ch` → `in_channels`, `out_ch` → `out_channels`, `kernel` → `kernel_size`.

4.4.6 Output Code Structure

Listing 4.4.10 ▶ Generated output code template

```

1  import torch
2  import torch.nn as nn
3
4
5  class <ModelName>(nn.Module):
6      def __init__(self):
7          super(<ModelName>, self).__init__()
8          # one line per parameterized layer node
9
10     def forward(self, x):
11         # one line per node in topological order
12         return <output_var>
13
14
15 if __name__ == '__main__':
16     device = torch.device('<device>')
17     model  = <ModelName>().to(device)
18     x      = torch.randn(<batch_size>,
19         ↪ <input_shape>).to(device)
20     print(model(x).shape)

```

4.5 Shape Inference

Definition 4.5.1 ▶ Shape Inference

The `shape_inference.py` module propagates tensor shapes from the input node along the topological order. For each layer type, specific rules are applied: Linear changes the last dimension, Conv2d computes spatial dimensions, Flatten merges dimensions, and so on.

When a dimension mismatch is detected, a `ShapeError` is reported with the line number of the node declaration, and compilation is halted. The `--show-shapes` flag displays each

node's output shape.

4.6 Graphviz Visualization

The `graph_viz.py` module produces Graphviz DOT output. Nodes are color-coded by layer type (green for input, blue for Linear, orange for Conv, purple for Normalization). The output node is marked with `peripheries=2` and labeled edges are shown with dashed lines. If shape inference is enabled, shapes are annotated on the nodes.

Listing 4.6.1 ▸ Generating and rendering a graph

```
1 python main.py -i input/inception.nng --graph-viz
   ↪ output/inception.dot
2 dot -Tpng output/inception.dot -o output/inception.png
```

4.7 Training Scaffold

If the `loss` key is set in the `config` block, the code generator produces a complete training loop including loss function, optimizer, and epoch loop. Supported loss functions: `CrossEntropyLoss`, `MSELoss`, `BCELoss`, `BCEWithLogitsLoss`, `L1Loss`, `NLLLoss`, `SmoothL1Loss`. Supported optimizers: `Adam`, `SGD`, `AdamW`, `RMSprop`.

4.8 Quantization Annotations

The universal `quant` parameter can be applied to any node. Three modes are supported: `"dynamic"`, `"static"`, and `"qat"`. QAT nodes are wrapped in `torch.quantization.QuantWrap`. When at least one node has a quantization annotation, `QuantStub` and `DeQuantStub` are added to `__init__` and `forward`.

4.9 ONNX Export

The `onnx_export.py` module instantiates the generated code and uses `torch.onnx.export` to produce an `.onnx` file with a dynamic batch axis and opset 17.

Listing 4.9.1 ▸ Exporting a model to ONNX

```
1 python main.py -i input/mlp.nng --onnx output/mlp.onnx
```

Evaluation

5.1 Test Suite

Definition 5.1.1 ▶ Test statistics

- Total tests: **55**
- Framework: `pytest` version 8.4.2
- Runtime: approximately 1.09 seconds

File	Tests	Coverage
<code>test_parser.py</code>	8	Grammar parses all 4 examples with zero syntax errors; config, label, shape
<code>test_listener.py</code>	14	4 valid inputs; 10 error conditions with <code>SystemExit</code>
<code>test_codegen.py</code>	19	nn class names, parameter values, forward patterns
<code>test_e2e.py</code>	14	Full compile + instantiate + forward pass with correct shape

Test execution command:

Listing 5.1.2 ▶ Running the test suite

```
1 python -m pytest tests/ -v
```

5.2 Forward Pass Results

Example 5.2.1 ▶ MLP

- Input: `tensor(784)` — Output: `(batch, 10)`
- Test: Softmax rows sum to 1 (`atol=1e-5`)

Example 5.2.2 ▶ ResBlock

- Input: `tensor(64, 32, 32)` — Output: `(batch, 65536)`
- Test: correct shape; deterministic output in `eval` mode

Example 5.2.3 ▶ InceptionBlock

- Input: `tensor(32, 56, 56)` — Output: `(batch, 48, 56, 56)`
- Branch A: 16 channels; Branch B: 32 channels; merged via `Concat(dim=1)` → 48 channels

Example 5.2.4 ▶ TransformerEncoder

- Input: `tensor(512, 128)` — Output: `(1024, 128)`
- Two implicit residual sub-layers without explicit `Residual()` nodes
- `MultiHeadAttn` with `embed_dim=128, num_heads=8, batch_first=True`

5.3 Variable Correctness Proof

Definition 5.3.1 ▶ Tensor non-overwrite proof

For each branching example, the input tensor's `data_ptr()` was traced through the forward computation:

- **ResBlock**: `x.data_ptr()` unchanged at the point `x = conv1 + x`
- **Inception**: `x.data_ptr()` identical when `convA1(x)` and `convB1(x)` are called
- **Transformer**: `x.data_ptr()` unchanged when `norm1(x + attn)` is called; `norm1.data_ptr()` unchanged when `norm2(norm1 + ff1)` is called

This follows directly from the invariant: `var_at[n]='x'` only when


```
out_degree=1.
```

5.4 Codebase Statistics

File/Directory	Lines
NNGraph.g4	56
custom_listener.py	303
code_generator.py	327
shape_inference.py	193
graph_viz.py	90
onnx_export.py	47
main.py	121
tests/ (4 files)	575
required_code_collection/	≈100
Total	≈1812

- Language: Python 3.14.3
- ANTLR: 4.13.2
- PyTorch: 2.10.0
- pytest: 8.4.2

Design Decisions

6.1 Listener vs. Visitor

The Listener pattern naturally suits state accumulation. The `exit*` methods can operate directly on `self.nodes` and `self.edges` without needing to return values through the tree.

6.2 `load_from_listener()` vs. `generate(traversal)`

Some compilers use `CodeGenerator.generate(traversal)`, which receives a post-order token list—suited for stack-based bytecode generation. For a graph compiler, the natural input is graph structure (nodes + edges) rather than a linear token stream. Therefore, `load_from_listener(listener)` is more direct.

6.3 Variable Naming with `node_id`

The language specification examples use single-letter or descriptive names. This compiler uses the actual node identifier because:

- It is deterministic and works for larger graphs without a counter
- It is readable—the variable `convA1` is clearer than `a`
- The mathematical output is identical

Conclusion and Future Work

7.1 Conclusion

NNGraph DSL provides a six-stage compiler pipeline from `.nng` to executable `nn.Module`. The compiler features an ANTLR4 grammar with 14 parser rules, 24 layer types, 8 validation rules, and a variable assignment algorithm that handles complex branching without tensor overwrite. The compiler has been validated with 55 tests across four example architectures.

7.2 Implemented Features

Feature	Description
Shape inference	Tensor shape propagation and mismatch detection at compile time (<code>--show-shapes</code>)
Graphviz visualization	Model graph output in DOT format (<code>--graph-viz</code>)
Training scaffold	Training loop generation with optimizer and loss from <code>config</code> block
Quantization	<code>quant</code> metadata for model quantization conversion
ONNX export	Model export to ONNX format (<code>--onnx</code>)

7.3 Future Work

Feature	Description
Named sub-graphs	Reusable macro blocks
Dynamic control flow	<code>if/else</code> branches within <code>forward</code>

References

1. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
2. Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.
3. NADER Framework: Neural Architecture Design via Representation (referenced in `NNGraph_DSL_Specification.md`).
4. Zhang, A. M. *amznotes LaTeX template*. github.com/alexmingzhang/latex-notes-template